Apply filters to SQL queries

Project description

In this Project we had potential threat that occurs in our database, and as we well know we had to monitorized our database to keep it up to date, so for this Project we use SQL queries By applying filtering techniques with WHERE, AND, OR, NOT, and LIKE, I was able to retrieve relevant information from the log_in_attempts and employees tables. As we see we can use SQL to support Cybersecurity incidents.

The `organization` database contains the following two tables:

- `log_in_attempts`
- `employees`

log_in_attempts

The `log_in_attempts` table has the following columns:

- `event_id`: The identification number assigned to each login event
- `username`: The username of the employee
- `login_date`: The date the login attempt was recorded
- `login_time`: The time the login attempt was recorded
- `country`: The country where the login attempt occurred
- `ip_address`: The IP address of that employee's machine
- `success`: The success of the login attempt; `FALSE` indicates a failed attempt

In the MariaDB shell, these columns are returned as:

```
+----------+----------+------------+------------+---------+-----------------+---------+
| event_id | username | login_date | login_time | country | ip_address      | success |
+----------+----------+------------+------------+---------+-----------------+---------+
```

employees

The `employees` table has the following columns:

- `employee_id`: The identification number assigned to each employee
- `device_id`: The identification number assigned to each device used by the employee
- `username`: The username of the employee
- `department`: The department the employee is in
- `office`: The office the employee is located in

In the MariaDB shell, these columns are returned as:



Retrieve after hours failed login attempts

SQL Query

```
SELECT *
FROM log_in_attempts
WHERE login_time > '18:00:00'
  AND success = 0;
```

Explanation

I wrote this query to find all login attempts that failed after normal business hours. I used the AND operator to combine two conditions: the login had to occur after **6:00 PM** and it had to be unsuccessful (success = 0). This helps identify suspicious login activity that happens when employees are less likely to be working.

Retrieve login attempts on specific dates

SQL Query

```
SELECT *
FROM log_in_attempts
WHERE login_date = '2022-05-09'
   OR login_date = '2022-05-08';
```

This query looks for all login attempts that happened on **May 8** or **May 9, 2022**. I used the OR operator so the query would return records from either date. This makes it easier to investigate activity that occurred around the time of a reported security incident.

Retrieve login attempts outside of Mexico

SQL Query

```
SELECT *
FROM log_in_attempts
WHERE NOT country LIKE 'MEX%';
```

Explanation

This query returns all login attempts that did not originate from Mexico. The `LIKE` operator, together with the `%` wildcard, matches both `MEX` and `MEXICO`. Adding the `NOT` operator excludes those records and returns login attempts from every other country. Using `LIKE` makes the query flexible when different values share the same prefix.

Retrieve employees in Marketing

SQL Query

```
SELECT *
FROM employees
WHERE department = 'Marketing'
  AND office LIKE 'East-%';
```

This query finds employees who work in the **Marketing** department and are located in the **East** building. I used the AND operator because both conditions must be true. I also used LIKE 'East-%' so the query includes every office in the East building, no matter what office number follows.

Retrieve employees in Finance or Sales

SQL Query

```
SELECT *
FROM employees
WHERE department = 'Finance'
   OR department = 'Sales';
```

I created this query to locate employees in either the **Finance** or **Sales** department. The OR operator allows the query to return employees from both departments because they all need the same security update. This makes it easy to identify everyone who should receive the update.

Retrieve all employees not in IT

SQL Query

```
SELECT *
FROM employees
WHERE NOT department = 'Information Technology';
```

This query returns all employees except those in the **Information Technology** department. I used the NOT operator because the IT team had already completed the security update, so only employees from the other departments needed to be included.

Summary

In this project, I used SQL to investigate potential security issues and identify employees whose devices required security updates. By applying filters, I was able to narrow down the data based on dates, times, locations, departments, and login results. This project strengthened my understanding of how SQL can be used in cybersecurity to quickly find relevant information, investigate suspicious activity, and support security operations more effectively.